

```
In [23]: import pickle
from sklearn.linear_model import LogisticRegression
import numpy as np
import os
from sklearn.preprocessing import StandardScaler
import pandas as pd
```

```
In [24]: os.getcwd()
```

```
Out[24]: '/Users/mariadecaro/final-project-part-1-maria-laia-victoria-bhoomika/agents/maria-laia-victoria-bhoomika'
```

```
In [25]: os.listdir("../")
```

```
Out[25]: ['dummy_fixed_prices_adaptive.py',
'.DS_Store',
'dummy_fixed_prices.py',
'__init__.py',
'__pycache__',
'maria-laia-victoria-bhoomika',
'static_prices_submission.csv',
'maria-laia-victoria-bhoomika.py']
```

```
In [26]: os.listdir("../../data")
```

```
Out[26]: ['train_prices_decisions_2025.csv',
'data_helpers_2025.ipynb',
'datafile1_2025.csv',
'datafile2_2025.csv',
'test_user_info_2025.csv']
```

reads in training data and convert item bought column to numeric for logistic regression (currently set as True/False)

```
In [27]: train_df = pd.read_csv("../../data/train_prices_decisions_2025.csv")
train_df
```

Out [27]:

	user_index	Covariate1	Covariate2	Covariate3	price_item	item_bought
0	0	1.396069	0.228525	9.790878	0.010000	True
1	1	5.201819	2.212083	10.549586	50.563709	True
2	2	5.471765	0.568564	12.706814	82.305243	True
3	3	3.002355	1.185335	5.513210	38.611238	True
4	4	1.337119	1.374958	6.298567	86.705523	False
...	...	...	...	...	...	...
49995	49995	6.921727	1.362905	6.257731	375.348066	False
49996	49996	1.706747	1.921548	12.139479	118.039923	False
49997	49997	5.270051	1.584286	8.507552	0.010000	True
49998	49998	2.447797	2.340020	5.929868	53.780429	False
49999	49999	4.073631	1.368670	23.828259	59.302147	True

50000 rows x 6 columns

```
In [28]: train_df['item_bought'] = train_df['item_bought'].astype(int)
train_df
```

Out [28]:

	user_index	Covariate1	Covariate2	Covariate3	price_item	item_bought
0	0	1.396069	0.228525	9.790878	0.010000	1
1	1	5.201819	2.212083	10.549586	50.563709	1
2	2	5.471765	0.568564	12.706814	82.305243	1
3	3	3.002355	1.185335	5.513210	38.611238	1
4	4	1.337119	1.374958	6.298567	86.705523	0
...	...	...	...	...	...	...
49995	49995	6.921727	1.362905	6.257731	375.348066	0
49996	49996	1.706747	1.921548	12.139479	118.039923	0
49997	49997	5.270051	1.584286	8.507552	0.010000	1
49998	49998	2.447797	2.340020	5.929868	53.780429	0
49999	49999	4.073631	1.368670	23.828259	59.302147	1

50000 rows x 6 columns

seperates features x and y, fits the scaler so that all are compared at the same level/scale, and fits the logistic regression model

```
In [29]: x = train_df[["price_item", "Covariate1", "Covariate2", "Covariate3"]]
y = train_df["item_bought"]

#this is for later so that we know the exact column order for predictions!
training_feature_columns = ["price_item", "Covariate1", "Covariate2",
"Covariate3"]
```

```
In [30]: scaler = StandardScaler()
scaler.fit(x)

x_scaled = scaler.transform(x)

model = LogisticRegression(max_iter=500)
model.fit(x_scaled, y)
```

```
Out[30]: LogisticRegression
LogisticRegression(max_iter=500)
```

save the model and scaler as pickle files (important to save the scaler since the model we trained is trained on these scaled features and not raw features so when we make the predictions on new customers we should apply the exact same scaling)

```
In [31]: with open("../agents/maria-laia-victoria-bhoomika/scaler.pkl", "wb") as f:
pickle.dump(scaler, f)

with open("../agents/maria-laia-victoria-bhoomika/demand_model.pkl",
"wb") as f:
pickle.dump(model, f)
```

open the pickle files, make sure that all is working, and just do a fake test run on our training data so that we know we can access and run these pickle files in the future!

```
In [32]: with open("../agents/maria-laia-victoria-bhoomika/scaler.pkl", "rb") as f:
loaded_scaler = pickle.load(f)

with open("../agents/maria-laia-victoria-bhoomika/demand_model.pkl",
"rb") as f:
loaded_model = pickle.load(f)

x_raw = x.iloc[[0]]
x_scaled = loaded_scaler.transform(x_raw)

pred_prob = loaded_model.predict_proba(x_scaled)[0, 1]

print("check of predicted probability of purchase:",
round(float(pred_prob), 4))
```

check of predicted probability of purchase: 0.9902

helper to compute the best prices given one user's covariates. here i tried as many prices and chose the revenue-maximizing price

right now i am setting a place holder of 200 for the ngrid and will test and eval later

i am also setting the min and max to that found in the training set

```
In [33]: price_min = train_df['price_item'].min()
price_max = train_df['price_item'].max()
print(price_min, price_max)
n_grid = 100

price_grid = np.linspace(price_min, price_max, n_grid)
```

0.01 452.339045127419

```
In [34]: def best_price_and_expected_rev_for_one_user(
        loaded_scaler,
        loaded_model,
        user_covariate_values_no_price
    ):

        covariate_df = pd.DataFrame([user_covariate_values_no_price.values]
        * len(price_grid), columns=user_covariate_values_no_price.index)
        covariate_df.insert(0, "price_item", price_grid)

        x_raw = covariate_df[training_feature_columns]
        x_scaled = loaded_scaler.transform(x_raw)

        purchase_prob = loaded_model.predict_proba(x_scaled)[:, 1]

        expected_revenue = price_grid * purchase_prob

        idx = int(np.argmax(expected_revenue))
        return float(price_grid[idx]), float(expected_revenue[idx])
```

now here I load the test users, loop through, and write the result to the csv that will go through the autograder

```
In [35]: test_df = pd.read_csv("../data/test_user_info_2025.csv")
test_df
```

Out [35]:

	user_index	Covariate1	Covariate2	Covariate3
0	50000	0.584638	5.706175	9.668797
1	50001	3.548108	7.374092	5.104997
2	50002	1.713915	4.704735	8.012817
3	50003	5.767208	1.807943	6.023578
4	50004	1.021004	2.912415	11.899356
...	...	...	...	...
49995	99995	0.915252	0.174688	4.327159
49996	99996	8.611839	0.893256	4.429134
49997	99997	1.222712	3.669642	4.454837
49998	99998	1.160867	3.250831	5.595728
49999	99999	2.981097	1.565321	4.421446

50000 rows x 4 columns

```

In [36]: test_covariate_cols = ["Covariate1", "Covariate2", "Covariate3"]

rows = []
for _, user in test_df.iterrows():
    covariate_series = user[test_covariate_cols]
    best_price, best_expected_revenue =
best_price_and_expected_rev_for_one_user(
    loaded_scaler=loaded_scaler,
    loaded_model=loaded_model,
    user_covariate_values_no_price = covariate_series
)
    rows.append({
        "user_index": int(user["user_index"]),
        "price_item": best_price,
        "expected_revenue": best_expected_revenue
    })

all_predicted_prices = pd.DataFrame(rows, columns=["user_index",
"price_item", "expected_revenue"])
all_predicted_prices.to_csv("../agents/static_prices_submission.csv",
index=False)
all_predicted_prices.head()

```

Out [36]:

	user_index	price_item	expected_revenue
0	50000	27.423882	14.025869
1	50001	31.992862	15.261783
2	50002	36.561842	20.049545
3	50003	59.406743	46.823905
4	50004	41.130822	26.434236

calculated min, max, and average expected revenue from the training data to be used in the agent

```
In [37]: import numpy as np

train_covariate_cols = ["Covariate1", "Covariate2", "Covariate3"]

def best_price_and_expected_rev_for_one_user(covariate_series):
    #matched the logic from our agent file
    x_raw = np.zeros((len(price_grid), 4))
    x_raw[:, 0] = price_grid
    x_raw[:, 1] = covariate_series["Covariate1"]
    x_raw[:, 2] = covariate_series["Covariate2"]
    x_raw[:, 3] = covariate_series["Covariate3"]

    feature_columns = ["price_item", "Covariate1", "Covariate2",
"Covariate3"]
    x_raw = pd.DataFrame(x_raw, columns=feature_columns)

    x_scaled = loaded_scaler.transform(x_raw)
    probs = loaded_model.predict_proba(x_scaled)[:, 1]
    evs = price_grid * probs

    idx = int(np.argmax(evs))
    return float(price_grid[idx]), float(evs[idx])

#use the same price_grid as our .py file
price_grid = np.linspace(0.01, 452.339045127419, 100)

evs = []
for _, user in train_df.iterrows():
    covariate_series = user[train_covariate_cols]
    _, ev_star = best_price_and_expected_rev_for_one_user(covariate_series)
    evs.append(ev_star)

evs = np.array(evs)
print("First 10 EV*:", evs[:10])
print("Min EV*:", evs.min())
print("Max EV*:", evs.max())
print("Avg EV*:", evs.mean())
```

First 10 EV*: [38.25146398 45.1763865 55.49725906 38.03306816 31.23630531 1
1.00141786
47.64231999 33.22301961 19.48130483 60.14469399]
Min EV*: 0.21617659312216334
Max EV*: 273.34926783754275
Avg EV*: 33.40450734169381

now I want to decide what n-grid value would be the best value for our model

```
In [38]: price_min = train_df['price_item'].min()
price_max = train_df['price_item'].max()
print(price_min, price_max)
#here are the different options to test
ng_choices = [50, 100, 200, 400]
#the number of test users to sample for the measurement
sample_users = 1500
```

0.01 452.339045127419

```
In [39]: def best_price_for_user_with_dynamic_grid(ngrid, cov_series):
global price_grid
price_grid = np.linspace(price_min, price_max, int(ngrid))
return best_price_and_expected_rev_for_one_user(cov_series)
```

```
In [40]: import time

import warnings
warnings.filterwarnings("ignore")

#I used chatgpt here to help print out a fancy looking grid with the n_grid
outputs – but did the code and logic myself! Just wanted some help from
chat to make the output look nice!
def evaluate_ngrid_list(ng_list, sample_n=1500):
    sample = test_df.iloc[:sample_n].copy()
    results = {}

    for N in ng_list:
        t0 = time.time()
        evs, prices = [], []
        for _, r in sample.iterrows():
            cov = r[test_covariate_cols]
            p_star, ev_star = best_price_for_user_with_dynamic_grid(N, cov)
            evs.append(ev_star)
            prices.append(p_star)
        elapsed = time.time() - t0
        evs = np.array(evs)
        results[N] = {"evs": evs, "prices": np.array(prices), "time_s":
elapsed, "avg_ev": float(evs.mean())}

    all_evs = np.column_stack([results[N]["evs"] for N in ng_list])
    per_user_best = all_evs.max(axis=1)

    print("N_GRID | avg_EV | avg_regret% | max_regret% | time_s")
    print("-----+-----+-----+-----+-----")
    for N in ng_list:
        evs_N = results[N]["evs"]
        regret = (per_user_best - evs_N) / (per_user_best + 1e-12)
        print(f"{N:<7} | {results[N]['avg_ev']:<11.6f} |
{100*regret.mean():<11.4f} | {100*regret.max():<11.4f} | {results[N]
['time_s']:.2f}")

    return results

_ = evaluate_ngrid_list(ng_choices, sample_n=sample_users)
```

N_GRID	avg_EV	avg_regret%	max_regret%	time_s
50	33.272107	0.5331	3.1109	0.86
100	33.386078	0.1413	0.8493	0.91
200	33.419242	0.0261	0.2204	0.94
400	33.426190	0.0020	0.0381	0.89

based on the information above, it shows that the n-grid value of 100 is optimal, since it returns the same average ev as the other values and does so within the quickest time out of the other n-grid options.

all the grid sizes give the EXACT same expected revenue which means even a coarse grid of 50 points is already good enough to find the max. However, the 100 n-grid seems

to be performing jsut a tad bit faster.

Regret is 0% across the board which means every N correctly finds the optimal price. Runtime increases as N gets larger which means that there is no benefit for using bigger grids.

this next section is just a sanity-check visual. here we grab one example user and run our trained model on a smooth range of prices and plot:

1. the predicted purchase probability at each price (the demand curve)
2. the expected revenue at each price (the revenue curve)

this lets us visually confirm that the model behaves like normal. normal is defined as:

1. demand should drop as price increases, revenue should peak somewhere in the middle.
2. If the curves look smooth and the best price makes sense, our demand model is working.

```

In [41]: import matplotlib.pyplot as plt

example_covs = test_df.iloc[0][["Covariate1", "Covariate2",
"Covariate3"]].values

def predict_prob(covs, price):
    row = pd.DataFrame([{"price_item": price,
"Covariate1": covs[0],
"Covariate2": covs[1],
"Covariate3": covs[2],
}])
    Xs = scaler.transform(row)
    return model.predict_proba(Xs)[0, 1]

def expected_revenue(covs, price):
    return price * predict_prob(covs, price)

#evaluate curves on a fine grid
PLOT_GRID = np.linspace(0.01, 452.339, 400)

probs = np.array([predict_prob(example_covs, p) for p in PLOT_GRID])
evs = np.array([expected_revenue(example_covs, p) for p in PLOT_GRID])

best_idx = evs.argmax()
best_price = PLOT_GRID[best_idx]
best_ev = evs[best_idx]

plt.figure(figsize=(10,5))

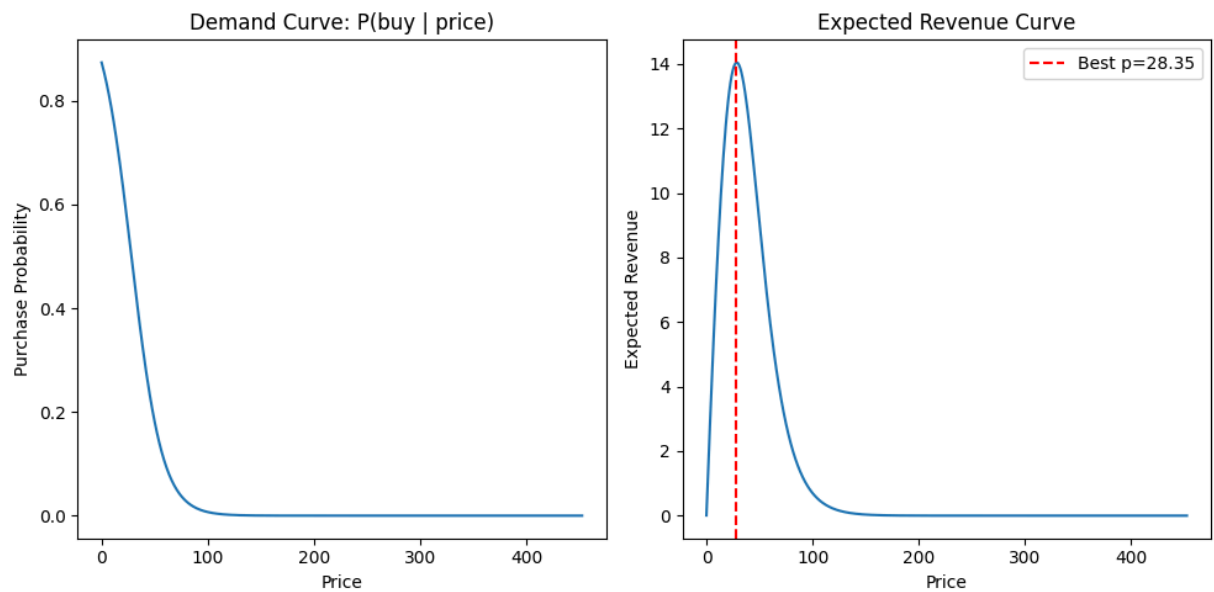
plt.subplot(1,2,1)
plt.plot(PLOT_GRID, probs)
plt.title("Demand Curve: P(buy | price)")
plt.xlabel("Price")
plt.ylabel("Purchase Probability")

plt.subplot(1,2,2)
plt.plot(PLOT_GRID, evs)
plt.axvline(best_price, color='red', linestyle='--', label=f"Best p=
{best_price:.2f}")
plt.title("Expected Revenue Curve")
plt.xlabel("Price")
plt.ylabel("Expected Revenue")
plt.legend()

plt.tight_layout()
plt.show()

print(f"Best price = {best_price:.4f}, Expected Revenue = {best_ev:.6f}")

```



Best price = 28.3514, Expected Revenue = 14.046775

here I generated the histogram of revenue maximizing prices to identify the specific price range where our demand model most frequently places the optimal price which allows us to focus our price search more efficiently! for this i used chatgpt to help with printing out the quartiles and generating the graph

```
In [42]: import warnings
warnings.filterwarnings("ignore")

temp_grid = np.linspace(0.01, price_max, 300)

best_prices = []

for x_row in x.values:
    #repeat the row across the grid
    x_grid = np.tile(x_row, (len(temp_grid), 1))

    #replace price column with the grid
    x_grid[:, 0] = temp_grid

    feature_columns = ["price_item", "Covariate1", "Covariate2",
"Covariate3"]
    x_raw = pd.DataFrame(x_grid, columns=feature_columns)

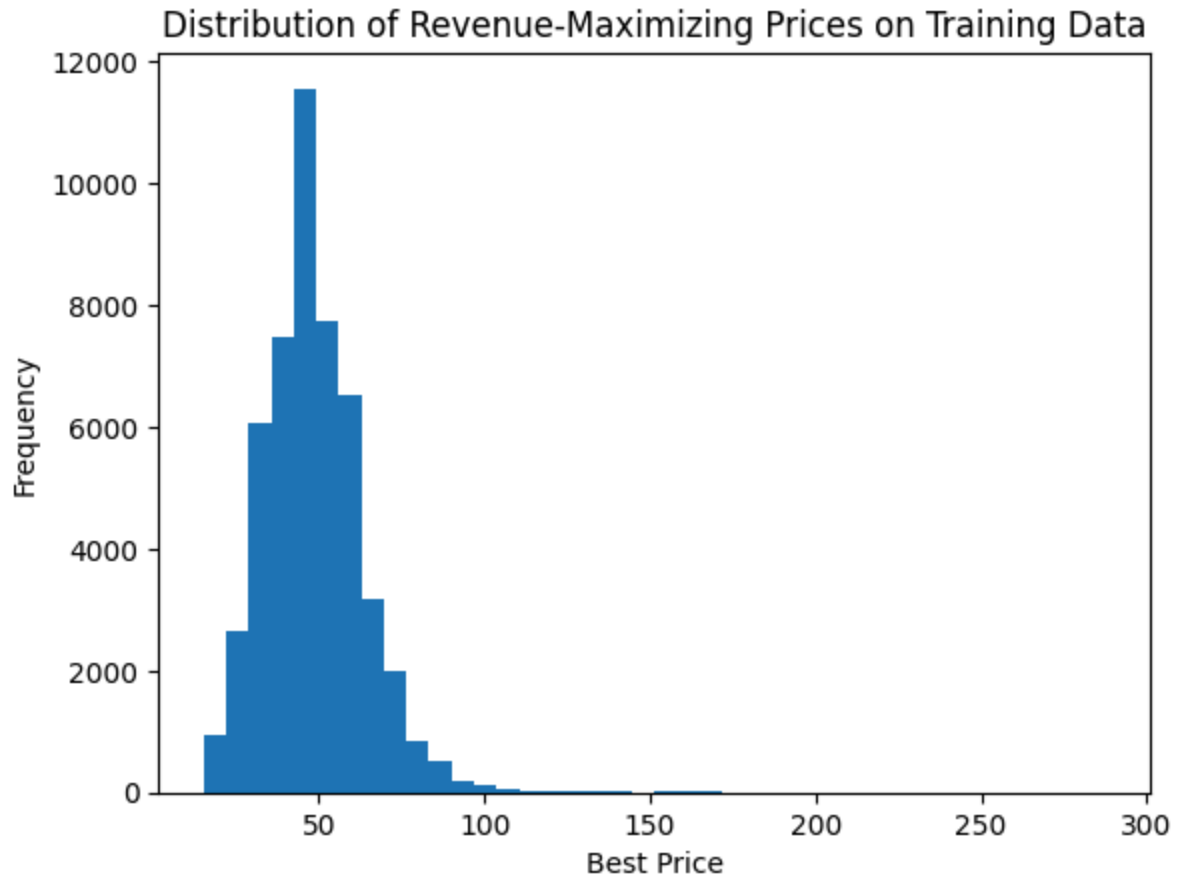
    x_scaled = loaded_scaler.transform(x_grid)
    probs = loaded_model.predict_proba(x_scaled)[:, 1]

    revenue = probs * temp_grid
    best_prices.append(temp_grid[np.argmax(revenue)])

#convert to array for quantiles
best_prices = np.array(best_prices)

plt.hist(best_prices, bins=40)
plt.title("Distribution of Revenue-Maximizing Prices on Training Data")
plt.xlabel("Best Price")
plt.ylabel("Frequency")
plt.show()

q25, q50, q75, q90 = np.percentile(best_prices, [25, 50, 75, 90])
print("Best Price Distribution Quantiles:")
print(f" 25th percentile: {q25:.2f}")
print(f" 50th percentile (median): {q50:.2f}")
print(f" 75th percentile: {q75:.2f}")
print(f" 90th percentile: {q90:.2f}")
```



Best Price Distribution Quantiles:

25th percentile: 37.83

50th percentile (median): 46.91

75th percentile: 55.98

90th percentile: 66.57

here the quartiles show that nearly all optimal prices fall between about 38 and 67... so we can concentrate our price grid densely in this region to improve accuracy while keeping the agent computationally efficient